GenieX on Snapdragon X — Prerequisites & Setup Guide

Nov 4, 2026 · @Vaibhav

Overview

Image-to-text (ITT) and video-to-text (VTT) now run on the Snapdragon X NPU through GenieX, after an ARM64 Python setup and a Hexagon NPU driver update. This guide lists what a machine needs, how to set it up, and every issue we hit on the way.

Item	Value
Device	Lenovo ThinkCentre neo 50q QC (13C8000YUS), hostname SHINGL143
Chip	Snapdragon X, X1-26-100 (8 cores, 45 TOPS Hexagon NPU). Not an X Elite.
OS	Windows 11 ARM64 (build 26200)
Inference SDK	GenieX 0.10.0 (Python bindings)
Runtimes	llama.cpp (GGUF models on NPU, hybrid, GPU, CPU) and QAIRT (Qualcomm AI Hub pre-compiled bundles, NPU only)
Working NPU driver	Hexagon NPU 30.0.224.10000 (dated 2026-07-24)

GenieX replaces the older QAIRT `genie-app.exe` + `genie-app-script.txt` flow. Qualcomm (Khoi Vo, AI Hub Slack) recommended it for both LLMs and VLMs.

Prerequisites

A machine is ready when every row below checks out. The two that blocked us longest were ARM64 Python and the NPU driver.

Requirement	Needed	How to check
Snapdragon device	Windows on ARM64 (X Elite, X Plus or X)	`echo %PROCESSOR_ARCHITECTURE%` prints `ARM64`
Python	Native ARM64 build, 3.11 or newer (we use 3.12.10). x64 Python and any 3.10 build will not work.	`python -c "import platform; print(platform.machine())"` prints `ARM64`
Virtual environment	Created from the ARM64 interpreter	`py -3.12-arm64 -m venv venv`
GenieX	`pip install geniex` (0.10.0)	Install log shows `geniex-sdk-windows-arm64` downloaded
NPU driver	Recent Hexagon NPU driver from Qualcomm Software Center (30.0.224.10000 works; 30.0.140.1000 does not)	PowerShell driver query (see Setup)
ffmpeg	Any recent build in PATH (VTT only)	`ffmpeg -version`
Disk	About 15 GB free for all test models in `C:\Users\<user>\.cache\geniex\models`	
RAM	16 GB recommended for 4B to 7B VLMs	
Network	Hugging Face and Qualcomm S3 reachable for first model download	

Models are downloaded automatically on first use and cached outside the venv, so recreating a venv never re-downloads them.

Setup steps

Run these in order on a fresh Snapdragon machine. Commands marked Admin need an Administrator PowerShell.

1. **Confirm the chip.** Run `Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores` in PowerShell. Record the exact model (e.g. Snapdragon X X1-26-100), since QAIRT bundles are chip-specific.

2. **Install ARM64 Python 3.12.**

   ```
   winget install --id Python.Python.3.12 --architecture arm64
   ```

   Open a new terminal, then run `py --list`. It must show `-3.12-arm64`.

3. **Create the venv from the ARM64 interpreter.** A plain `python -m venv` picks whichever Python is first on PATH, which here was x64 3.10.

   ```
   cd C:\Users\shi-lab\Documents
   py -3.12-arm64 -m venv venv
   venv\Scripts\activate
   python -c "import platform; print(platform.machine())"
   ```

   Continue only if it prints `ARM64`.

4. **Install GenieX.**

   ```
   python -m pip install --upgrade pip
   pip install geniex
   ```

   The pip package is the Python bindings only. The `geniex` CLI is a separate installer from the GitHub Releases page and is not needed for these scripts.

5. **Update the NPU driver (Admin).** Download the latest Hexagon NPU Driver (`Qualcomm_HND`, package 1.2.4667.8400) from Qualcomm Software Center with a Qualcomm ID, install it, and reboot. Windows Update and Lenovo packages did not include a newer NPU driver for this machine.

6. **Verify the driver (Admin).**

   ```
   Get-CimInstance Win32_PnPSignedDriver | Where-Object { $_.DeviceName -
   like "*Hexagon*" } | Select-Object DeviceName, DriverVersion, DriverDate
   ```

   Expected: `30.0.224.10000` or newer. Filter on `*Hexagon*`, not `*NPU*`, which also matches "USB Input Device".

7. **Install ffmpeg (VTT only).**

   ```
   winget install Gyan.FFmpeg
   ```

   Open a new terminal and check `ffmpeg -version`. Without internet or admin, copy `ffmpeg.exe` into the working folder instead.

8. **Smoke test.** Run the ITT script (next section) on `cpu`, then `npu`. A description plus a tok/s line means the stack works.

Running workloads

All scripts run from the ARM64 venv. `device_map` picks the compute unit: `npu` (pure Hexagon), `hybrid` (NPU + CPU, the llama.cpp default when omitted), `gpu` (Adreno, via OpenCL), `cpu`, or `qairt` (AI Hub bundles, NPU only).

Image-to-text (ITT). One image, one load, one `generate()`:

```
from geniex import AutoModelForCausalLM

MODEL = "unsloth/Qwen3-VL-4B-Instruct-GGUF"
IMAGE =
r"...\genai_services\image_to_text\temp\data\Images\astronaut_horse.png"
DEVICE = "npu"

model = AutoModelForCausalLM.from_pretrained(MODEL, precision="Q4_0",
device_map=DEVICE)
messages = [{"role": "user", "content": [
    {"type": "image", "image": IMAGE},
    {"type": "text", "text": "Describe the image."},
]}]
prompt = model.tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
out = model.generate(prompt, images=[IMAGE], max_new_tokens=256)
print(out.text)
print(f"[{DEVICE}] {out.profile.generated_tokens} tok,
{out.profile.decode_speed:.1f} tok/s")
model.close()
```

Video-to-text (VTT). Same flow as the Intel `video_to_text/intel/trigger_test.py`: ffmpeg extracts all frames (`-q:v 2`), 1.25% are kept evenly spaced, prompt "Describe this video in detail.", `max_new_tokens=1024`. Three changes for GenieX:

- GenieX has no `{"type": "video"}` input, so each selected frame becomes an `{"type": "image"}` entry and the list goes to `generate(..., images=frames)`.
- The `file://` prefix from `get_frame_list` is stripped; GenieX takes plain paths.
- `processor.apply_chat_template` becomes `model.tokenizer.apply_chat_template`.

Model × device comparison. `npu_test_suite.py` runs each GGUF model (Qwen3-0.6B text, Qwen3.5-0.8B, Qwen3.5-2B, Qwen3-VL-4B, Qwen2.5-VL-7B) on all four devices, optionally the QAIRT bundles, each test in its own process. It checks VLM answers for

expected keywords, prints a tok/s grid and writes two CSVs.

```
python npu_test_suite.py --image "<path>\astronaut_horse.png"
python npu_test_suite.py --models vlm2b vlm4b --devices npu gpu --image "
<path>"
python npu_test_suite.py --qairt --image "<path>"
```

Results so far

The NPU now runs both text and vision models, but output quality on the NPU must be checked: Qwen3-VL-4B was fast there and described the wrong content. The full model × device grid from `npu_test_suite.py` is still to be run.

Workload	Model (runtime)	Device	Decode speed (tok/s)	Output quality
Text	Qwen3-0.6B Q4_0 (llama.cpp)	cpu	148.2	OK
Text	Qwen3-0.6B Q4_0 (llama.cpp)	npu	73.4	OK
Text	Qwen3-0.6B Q4_0 (llama.cpp)	hybrid	67.5	OK
Text	Qwen3-0.6B Q4_0 (llama.cpp)	gpu	54.8	OK
ITT	Qwen3.5-2B Q4_0 (llama.cpp)	hybrid	27.5	Correct: astronaut on a green horse
ITT	Qwen3-VL-4B Q4_0 (llama.cpp)	npu	12.8	Wrong: no astronaut, invented people with surfboards
VTT	Qwen3-VL-4B Q4_0 (llama.cpp), 3 frames of a 5.7 s golf clip	npu	14.7 (340 tokens, 26.8 s)	Partly wrong: read the frames as one "still image" with mirrored copies
Text	Qwen3-4B w4a16 (QAIRT, X Elite bundle)	qairt	1.0 (load 17 s)	OK text, unusably slow

Workload	Model (runtime)	Device	Decode speed (tok/s)	Output quality
ITT	Qwen2.5-VL-7B w4a16 (QAIRT, X Elite bundle)	qairt	Did not finish	Run stopped by hand

For tiny models the CPU is fastest; the NPU should pull ahead as models grow. The CPU and GPU rows for Qwen3-0.6B were measured on the old driver; the rest on 30.0.224.10000.

Issues and fixes

Two root causes explain most failures: Python running as x64 under emulation, and an NPU driver from February 2025. Issues are listed in the order we hit them.

#	Symptom	Cause	Fix
1	`pip install geniex` fails: `Unsupported platform ('win32', 'amd64')`	Python was the x64 build, running under emulation	Install ARM64 Python, create the venv with `py -3.12-arm64 -m venv`
2	`ERROR: Invalid requirement: '#'`	A `# comment` pasted into cmd, which has no inline comments	Drop the comment
3	`'geniex' is not recognized`	The pip package has no CLI; the CLI is a separate Windows ARM64 installer	Use the Python API (or install the CLI from GitHub Releases)
4	Qwen3-4B QAIRT: `OSError: exception: stack overflow` in `geniex_llm_create`	X Elite bundle (`soc_model 60`) on an X1-26-100, plus the old NPU driver	Driver update (row 9); bundle now loads
5	Qwen2.5-VL-7B QAIRT: process dies, `0xC0000028` (STATUS_BAD_STACK) in `_create_vlm_handle`	Same as row 4; native error path crashes instead of returning an error	Same as row 4

#	Symptom	Cause	Fix
6	A 64 MB worker-thread stack did not help	Stack size was never the cause	Removed from the scripts
7	llama.cpp on `npu`: `GenieXError(-100001): Invalid input parameters or handle` (Q8_0 and Q4_0); `cpu` and `gpu` work	NPU driver 30.0.140.1000 (2025-02-15) too old for GenieX / QAIRT 2.45	Driver update (row 9)
8	Windows Update installed Lenovo firmware 1.0.0.65 and driver packages 26.9.0.9 / 26.9.1.4, but NPU driver stayed 30.0.140.1000	Lenovo packages carry no newer NPU driver for this model	Get the driver from Qualcomm instead
9	NPU working	Hexagon NPU Driver from Qualcomm Software Center installed	Driver now 30.0.224.10000 (2026-07-24)
10	`No Python at ...\Python312-arm64\python.exe`	ARM64 3.12 was removed while installing 3.10.11; a venv points at its base Python	Reinstall ARM64 3.12, recreate the venv
11	Wanted Python 3.10.11	Python.org ships no Windows ARM64 build before 3.11	Use 3.11+ ARM64 for GenieX; keep 3.10 x64 in its own venv if another tool needs it
12	New `itt_venv` hits row 1 again	Created with the x64 3.10 interpreter	Recreate with `py -3.12-arm64`
13	Every VLM test in the suite: `GenieXError(-201202): VLM prefix reuse failed`	A second `generate()` on the same VLM handle fails in GenieX 0.10.0	Load a fresh model per image or video

#	Symptom	Cause	Fix
14	VTT script: `NameError` for `output_folder_path`, `idx`, `AutoModelForCausalLM`; `processor` undefined	Code copied from the Intel function; imports cut off when pasting	Define the variables, keep the imports, use `model.tokenizer`
15	VTT: `FileNotFoundError: [WinError 2]` in `extract_frames_ffmpeg`	ffmpeg not installed	`winget install Gyan.FFmpeg`, new terminal
16	`findstr ... metadata.json`: cannot open	Command run outside the bundle folder	Use the full cache path

Limitations and open items

The NPU path works, but four limits affect how BenchSmart results from this device should be read.

- **No X1-26-100 bundles.** AI Hub QAIRT bundles exist only for the X Elite. They load here after the driver update but decode at about 1 tok/s. Qualcomm has no X1-26-100 device to test on.
- **NPU accuracy.** Qwen3-VL-4B on `npu` produced wrong descriptions for both ITT and VTT. A fast but wrong answer must be marked failed, not counted. The test suite flags answers missing the expected keywords (default `astronaut,horse`).
- **No video input type.** VTT frames go in as separate images, so the model may not see them as a sequence. Telling it the images are ordered frames changes the prompt versus Intel's "Describe this video in detail."; record that difference in the Qualcomm workload notes.
- **VLM reload per call.** Each image or video needs a fresh model load (row 13), so load time is measured separately from inference.

Labeling rules for reports

- Name the chip as "Snapdragon X X1-26-100", not X Elite.
- Record runtime and quantization per row: GGUF Q4_0 (llama.cpp) and w4a16 (QAIRT) are different schemes.
- `hybrid` is NPU + CPU; do not report it as NPU.
- Keep the machine plugged in and idle during runs.

Open items

- [ ] Run the full `npu_test_suite.py` grid on the new driver
- [ ] Re-run Qwen3-VL-4B ITT and VTT on `gpu` and `cpu` to confirm whether the wrong answers are NPU-specific
- [ ] Re-run VTT with the "ordered frames" prompt
- [ ] Test Qwen2.5-VL-7B through llama.cpp GGUF (`vlm7b_*` tests)
- [ ] Ask Qualcomm why the X Elite Qwen3-4B bundle runs at 1 tok/s on X1-26-100
- [ ] Pick the BenchSmart ITT/VTT model per RAM tier (candidates: Qwen3.5-2B, Qwen3-VL-4B)

Qualcomm communication and references

Qualcomm's three answers on the AI Hub Slack unblocked the setup; the 1 tok/s QAIRT result has not been reported back yet.

Our question	Qualcomm's answer (Khoi Vo)
Right workflow for Qwen2.5-VL-7B on device (genie-app script, custom-image preprocessing, VLM docs)	Use GenieX directly, or the GenieX Chat Windows app
NPU fails on X1-26-100; which driver, and is the chip supported?	Always use the latest drivers. No X1-26-100 device to verify; its NPU matches the X Elite, so try the `qualcomm-snapdragon-x-elite` chipset, with no guarantee every model works
Where to get a newer NPU driver	Qualcomm Software Center, `Qualcomm_HND`

References

- GenieX on GitHub (install, quickstart, Releases for the CLI installer)
- GenieX documentation (Python quickstart, FAQ on runtimes and devices)
- Qwen2.5-VL-7B-Instruct on Hugging Face (Qualcomm)
- AI Hub Models v0.57.0 release notes (models tested on llama.cpp and QAIRT)
- Hexagon NPU Driver, Qualcomm Software Center
- GenieX Chat Windows app

Local paths on SHINGL143: model cache `C:\Users\shi-lab\.cache\geniex\models`; venv `C:\Users\shi-lab\Documents\venv`; QAIRT SDK 2.45 `C:\Users\shi-lab\Downloads\v2.45.0.260326\`.